PROJECT: ASSESSING CUSTOMER CHURN USING MACHINE LEARNING

datalab

The telecommunications (telecom) sector in India is rapidly changing, with more and more telecom businesses being created and many customers deciding to switch between providers. "Churn" refers to the process where customers or subscribers stop using a company's services or products. Understanding the factors that influence keeping a customer as a client in predicting churn is crucial for telecom companies to enhance their service quality and customer satisfaction. As the data scientist on this project, you aim to explore the intricate dynamics of customer behavior and demographics in the Indian telecom sector in predicting customer churn, utilizing two comprehensive datasets from four major telecom partners: Airtel, Reliance Jio, Vodafone, and BSNL:

- `telecom_demographics.csv` contains information related to Indian customer demographics:

Variable	Description
`customer_id`	Unique identifier for each customer.
`telecom_partner`	The telecom partner associated with the customer.
`gender`	The gender of the customer.
`age`	The age of the customer.
`state`	The Indian state in which the customer is located.
`city`	The city in which the customer is located.
`pincode`	The pincode of the customer's location.
`registration_event`	When the customer registered with the telecom partner.
`num_dependents`	The number of dependents (e.g., children) the customer has.
`estimated_salary`	The customer's estimated salary.

- `telecom_usage` contains information about the usage patterns of Indian customers:

Variable	Description
`customer_id`	Unique identifier for each customer.
`calls_made`	The number of calls made by the customer.
`sms_sent`	The number of SMS messages sent by the customer.
`data_used`	The amount of data used by the customer.
`churn`	Binary variable indicating whether the customer has churned or not (1 = churned, 0 = not churned).

```python
# Import libraries and methods/functions
import pandas as pd
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import RidgeClassifier, LogisticRegression  # <-- FIXED:
Added LogisticRegression import
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix

# Start your code here!
# Load data
telco_demog = pd.read_csv('telecom_demographics.csv')
telco_usage = pd.read_csv('telecom_usage.csv')

# Join data
churn_df = telco_demog.merge(telco_usage, on='customer_id')

# Identify churn rate
churn_rate = churn_df['churn'].value_counts() / len(churn_df)
print(churn_rate)

# Identify categorical variables
print(churn_df.info())

# One Hot Encoding for categorical variables
# Optional: if you are familiar with date and time manipulation, try working with
the registration_event column to see if it improves your modeling
churn_df = pd.get_dummies(churn_df, columns=['telecom_partner', 'gender', 'state',
'city', 'registration_event'])

# Feature Scaling
scaler = StandardScaler()

# 'customer_id' is not a feature
features = churn_df.drop(['customer_id', 'churn'], axis=1)
features_scaled = scaler.fit_transform(features)

# Target variable
target = churn_df['churn']

# Splitting the dataset
X_train, X_test, y_train, y_test = train_test_split(features_scaled, target,
test_size=0.2, random_state=42)

# Instantiate the Logistic Regression
logreg = LogisticRegression(random_state=42)
logreg.fit(X_train, y_train)
```

```python
# Logistic Regression predictions
logreg_pred = logreg.predict(X_test)

# Logistic Regression evaluation
print(confusion_matrix(y_test, logreg_pred))
print(classification_report(y_test, logreg_pred))

# Instantiate the Random Forest model
rf = RandomForestClassifier(random_state=42)
rf.fit(X_train, y_train)

# Random Forest predictions
rf_pred = rf.predict(X_test)

# Random Forest evaluation
print(confusion_matrix(y_test, rf_pred))
print(classification_report(y_test, rf_pred))

# Which accuracy score is higher? Ridge or RandomForest
higher_accuracy = "RandomForest"
```

```
0    0.799538
1    0.200462
Name: churn, dtype: float64
<class 'pandas.core.frame.DataFrame'>
Int64Index: 6500 entries, 0 to 6499
Data columns (total 14 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   customer_id         6500 non-null   int64
 1   telecom_partner     6500 non-null   object
 2   gender              6500 non-null   object
 3   age                 6500 non-null   int64
 4   state               6500 non-null   object
 5   city                6500 non-null   object
 6   pincode             6500 non-null   int64
 7   registration_event  6500 non-null   object
 8   num_dependents      6500 non-null   int64
 9   estimated_salary    6500 non-null   int64
 10  calls_made          6500 non-null   int64
 11  sms_sent            6500 non-null   int64
 12  data_used           6500 non-null   int64
 13  churn               6500 non-null   int64
dtypes: int64(9), object(5)
```